Supplementary Material: Two-Stage Drone Task Auction Mechanism Design for Post-Earthquake Emergency Rescue

1 Auxiliary Procedures of GA-LNS

To ensure the execution efficiency and solution quality of the GA-LNS algorithm, this section explains the three core auxiliary procedures called in the algorithm.

1.1 TwoOpt

`TwoOpt` independently executes 2-opt edge exchange optimization for each trip, reducing flight distance by reversing subsequences within the trip. Its input is the trip list $\mathcal{R}$, and the output is the optimized trip list $\mathcal{R}_{new}$.

Algorithm S1 TwoOpt($\mathcal{R}$)

Require: Trip list $\mathcal{R} = \{\text{trip}_1, \text{trip}_2, \ldots, \text{trip}_m\}$
Ensure: Optimized trip list $\mathcal{R}_{new}$

1: **for** each trip $\in \mathcal{R}$ **do**
2: improved $\leftarrow$ True
3: **while** improved == True **do**
4: improved $\leftarrow$ False
5: **for** $i = 1$ **to** $|\text{trip}| - 1$ **do**
6: **for** $j = i + 1$ **to** $|\text{trip}|$ **do**
7: $\text{trip}_{new} \leftarrow (\text{trip}_1, \ldots, \text{trip}_{i-1}, \text{trip}_j, \text{trip}_{j-1}, \ldots,$
8: $\text{trip}_i, \text{trip}_{j+1}, \ldots, \text{trip}_{|\text{trip}|})$
9: **if** FlightDist(trip_{new}, p_0) < FlightDist(trip, p_0) **then**
10: trip $\leftarrow \text{trip}_{new}$
11: improved $\leftarrow$ True
12: **end if**
13: **end for**
14: **end for**
15: **end while**
16: **end for**
17: **return** $\mathcal{R}$

As shown in Algorithm S1, Line 1 traverses each trip in $\mathcal{R}$. Lines 2-14 form the outer while loop, repeatedly scanning until a complete scan yields no improvement. Lines 5-6 enumerate all possible edge exchange pairs (i, j); Line 7 completely reverses the subsequence from position i to j to generate a new trip; Lines 8-10 accept the exchange and mark improvement if the new trip has a smaller flight distance. Line 17 returns the optimized trip list.

1.2 ComputeReturnTime

`ComputeReturnTime` recalculates the return time of each trip after `TwoOpt` optimization. Since `TwoOpt` only changes the POI visiting order within the trip without changing the trip division, there is no need to call PS again. Its input is the trip list $\mathcal{R}$, and the output is the latest return time $t_{\max}$.

As shown in Algorithm S2, Lines 2-12 traverse each trip: Lines 3-8 accumulate flight distance and hover time within the trip; Line 9 adds the return distance; Line 10 calculates the time consumption of this trip; Line 11 accumulates the current trip time consumption to the total return time $t_{\max}$. This function does not change the trip division, only recalculates the time, and its semantics are consistent with the closing time calculation of a single trip in PS.

Algorithm S2 ComputeReturnTime($\mathcal{R}$)

Require: Trip list $\mathcal{R} = \{\text{trip}_1, \text{trip}_2, \ldots, \text{trip}_m\}$
Ensure: Latest return time $t_{\max}$

1: $t_{\max} \leftarrow 0$
2: **for** each trip $\in \mathcal{R}$ **do**
3: $\text{dist}_{total} \leftarrow 0$; $\text{hover}_{total} \leftarrow 0$; pos $\leftarrow p_0$
4: **for** each $p \in$ trip **do**
5: $\text{dist}_{total} \leftarrow \text{dist}_{total} + \text{dist}(\text{pos}, p)$
6: $\text{hover}_{total} \leftarrow \text{hover}_{total} + h_p$
7: pos $\leftarrow p$
8: **end for**
9: $\text{dist}_{total} \leftarrow \text{dist}_{total} + \text{dist}(\text{pos}, p_0)$
10: $t_{return} \leftarrow \text{dist}_{total}/V_i + \text{hover}_{total}$
11: $t_{\max} \leftarrow t_{\max} + t_{return}$
12: **end for**
13: **return** $t_{\max}$

1.3 Concatenate

`Concatenate` sequentially concatenates the trip list $\mathcal{R}$ into the global visiting sequence π for use in GA crossover and mutation operations.

Algorithm S3 Concatenate($\mathcal{R}$)

Require: Trip list $\mathcal{R} = \{\text{trip}_1, \text{trip}_2, \ldots, \text{trip}_m\}$
Ensure: Global visiting sequence π

1: $\pi \leftarrow []$
2: **for** each trip $\in \mathcal{R}$ **do**
3: **for** each $p \in$ trip **do**
4: π.append(p)
5: **end for**
6: **end for**
7: **return** π

As shown in Algorithm S3, Lines 2-6 append elements one by one through a double loop to avoid list nesting caused by whole-append. Line 7 returns the concatenated global visiting sequence π.

2 Theoretical Proofs

This section provides the proofs of incentive compatibility (IC), individual rationality (IR), and the approximation ratio.

2.1 Reverse VCG: IC and IR

Theorem S1 (Incentive Compatibility of Reverse VCG). *Under the reverse VCG mechanism, for any candidate package $\theta_i \in \mathcal{I}'$, if*

the task package is exogenously given, then truthful bidding by the drone is its weakly dominant strategy, i.e., the mechanism satisfies Incentive Compatibility.

Proof: Consider drone u_i with true cost c_i, and its bid submitted to the center is b_i. Let $b_i^* = c_i$ be the truthful bid, and $b_i' \neq c_i$ be an arbitrary misreported bid.

Define $\mathbf{x}^*$ as the optimal allocation plan when all drones submit truthful bids, with corresponding minimum social cost $W(\mathbf{x}^*)$; $\mathbf{x}'$ as the optimal allocation plan when drone u_i submits misreported bid b_i' while other drones submit truthful bids, with corresponding minimum social cost $W(\mathbf{x}')$; $\mathbf{x}^*_{-i}$ and $\mathbf{x}'_{-i}$ are respectively the optimal allocation plans after excluding drone u_i from the auction, with other drones' bids unchanged. Since the optimal solution after excluding u_i is independent of u_i's own bid, we have $W(\mathbf{x}^*_{-i}) = W(\mathbf{x}'_{-i})$.

The utility ψ_i^* of drone u_i under truthful bidding is:

$$\psi_i^* = p_i^* - b_i^* x_i^* = W(\mathbf{x}^*_{-i}) - W(\mathbf{x}^*) \tag{1}$$

The utility ψ_i' under misreported bid $b_i' (\neq b_i^*)$ is:

$$\psi_i' = p_i' - b_i^* x_i' = W(\mathbf{x}'_{-i}) - W(\mathbf{x}') + b_i' x_i' - b_i^* x_i' \tag{2}$$

Subtracting the two equations and using $W(\mathbf{x}^*_{-i}) = W(\mathbf{x}'_{-i})$, we get:

$$\begin{aligned}
\psi_i' - \psi_i^* &= [W(\mathbf{x}'_{-i}) - W(\mathbf{x}') + b_i' x_i' - b_i^* x_i'] - [W(\mathbf{x}^*_{-i}) - W(\mathbf{x}^*)] \\
&= W(\mathbf{x}^*) - W(\mathbf{x}') + b_i' x_i' - b_i^* x_i' \\
&= \sum_{\theta_i \in \mathcal{I}'} b_i^* x_i^* - \left(\sum_{\theta_j \in \mathcal{I}', j \neq i} b_j^* x_j' + b_i' x_i' \right) + b_i' x_i' - b_i^* x_i' \\
&= \sum_{\theta_i \in \mathcal{I}'} b_i^* x_i^* - \sum_{\theta_j \in \mathcal{I}', j \neq i} b_j^* x_j' - b_i' x_i' + b_i' x_i' - b_i^* x_i' \\
&= \sum_{\theta_i \in \mathcal{I}'} b_i^* x_i^* - \sum_{\theta_j \in \mathcal{I}', j \neq i} b_j^* x_j' - b_i^* x_i' \\
&= \sum_{\theta_i \in \mathcal{I}'} b_i^* x_i^* - \left(\sum_{\theta_j \in \mathcal{I}', j \neq i} b_j^* x_j' + b_i^* x_i' \right) \\
&= \sum_{\theta_i \in \mathcal{I}'} b_i^* x_i^* - \sum_{\theta_i \in \mathcal{I}'} b_i^* x_i'
\end{aligned}$$

Because $\mathbf{x}^*$ is the optimal solution to the problem of minimizing social cost when drone u_i submits truthful bid, for any other feasible solution $\mathbf{x}'$ we have:

$$\sum_{\theta_i \in \mathcal{I}'} b_i^* x_i^* \leq \sum_{\theta_i \in \mathcal{I}'} b_i^* x_i' \tag{3}$$

Thus $\psi_i^* \geq \psi_i'$, and the mechanism designed in this paper satisfies Incentive Compatibility.

Theorem S2 (Individual Rationality of Reverse VCG). *Under truthful bidding, the utility of any winning candidate package is non-negative.*

Proof: Individual rationality requires proving that if a drone participates in the task, its utility is non-negative, i.e., $\psi_i^* \geq 0$. From the above utility formula:

$$\psi_i^* = p_i^* - b_i^* x_i^* = W(\mathbf{x}^*_{-i}) - W(\mathbf{x}^*)$$

Since the objective function is to find the minimum, removing drone u_i shrinks the feasible region, so the optimal social cost cannot decrease further, thus $W(\mathbf{x}^*_{-i}) \geq W(\mathbf{x}^*)$, i.e., $\psi_i^* \geq 0$, and the mechanism satisfies Individual Rationality.

2.2 GCPM: IC and IR

Theorem S3 (Incentive Compatibility of GCPM). *Under the GCPM mechanism, if the task package is exogenously given and the true cost does not exceed the bid upper bound $U_{\max}$, then truthful bidding $b_i = c_i$ is a weakly dominant strategy, i.e., the mechanism satisfies Incentive Compatibility.*

Proof: Consider any drone u_i, with true cost c_i. Let its utility under truthful bidding $b_i = c_i$ be ψ_i^*, and its utility under misreported bid $b_i \neq c_i$ be ψ_i'. Two cases are discussed:

(1) u_i wins under truthful bidding: (a) If u_i still wins under misreported bidding, then according to the critical value payment rule of GCPM-PAY, the payment price p_i depends only on the bids of other participants and is independent of u_i's own bid. Therefore $p_i' = p_i$, and the utility $\psi_i' = p_i' - c_i = \psi_i^*$, unchanged; (b) If misreported high bid $b_i' > c_i$ causes u_i to lose, then $\psi_i' = 0$. Since $\psi_i^* \geq 0$ under truthful bidding, $\psi_i' \leq \psi_i^*$; (c) If misreported low bid $b_i' < c_i$: Since the marginal cost-effectiveness $\rho_i(\mathcal{P}') = b_i / |\mathcal{P}_i \setminus \mathcal{P}'|$ decreases as the bid decreases, the priority of u_i in greedy selection only increases, not decreases. Therefore u_i necessarily also wins under the misreported low bid, falling into subcase (a).

(2) u_i loses under truthful bidding: (a) If u_i still loses under misreported bidding, then $\psi_i' = \psi_i^* = 0$, unchanged; (b) If misreported low bid $b_i' < c_i$ causes u_i to win: Let the critical price calculated by GCPM-PAY be p_i'. Since u_i loses under truthful bid c_i, according to the definition of critical price, we must have $p_i' \leq c_i$ (otherwise if $p_i' > c_i$, then truthful bid c_i is still lower than the critical value, and the greedy algorithm should select u_i, contradicting the loss). Therefore the utility after misreported winning is $\psi_i' = p_i' - c_i \leq 0 = \psi_i^*$; (c) If misreported high bid $b_i' > c_i$: The marginal cost-effectiveness $\rho_i(\mathcal{P}')$ increases as the bid increases, and the priority of u_i in greedy selection only decreases, not increases. Therefore u_i cannot win under misreported high bid, falling into subcase (a).

In summary, any misreporting behavior cannot improve the drone's utility, i.e., $\psi_i' \leq \psi_i^*$, so GCPM satisfies Incentive Compatibility.

Theorem S4 (Individual Rationality of GCPM). *Under truthful bidding, the utility of any winning drone is non-negative.*

Proof: For any winning candidate package θ_i, its payment price p_i is determined by the GCPM-PAY algorithm. If θ_i is the sole coverer of some POIs, the algorithm sets $p_i = U_{\max}$. Since it is assumed earlier that $c_i \leq U_{\max}$, the utility $\psi_i = p_i - c_i \geq 0$. Otherwise, the algorithm determines the critical price p_i of θ_i through binary search. According to the definition of critical price, θ_i is selected when the bid is lower than p_i, and loses when higher than p_i. Since θ_i wins under truthful bid c_i, we must have $c_i \leq p_i$. Therefore the utility $\psi_i = p_i - c_i \geq 0$. The utility of non-winning candidate packages is 0. Therefore the GCPM mechanism satisfies Individual Rationality.

2.3 Approximation Ratio of GCPM

Theorem S5 (Approximation Ratio). *Let C_{greedy} be the sum of bids of winning packages selected by the greedy algorithm, and C_{opt}*

be the optimal sum of bids of integer programming (18). Let $N = |\mathcal{P}|$. *Then:* $C_{greedy} \le H_N \cdot C_{opt}$, *where* $H_N = 1 + 1/2 + \cdots + 1/N$.

Proof: Let the package selected by the greedy algorithm in the t-th round be S_t, the number of newly covered POIs be m_t, and the number of remaining uncovered POIs at the beginning of this round be n_t. Let the optimal cover be O^*, whose total cost C_{opt} covers the current remaining n_t POIs. By the averaging principle, there must exist some $S \in O^*$ satisfying:

$$\frac{b_S}{|S \cap R_t|} \le \frac{C_{opt}}{n_t} \tag{4}$$

where R_t is the set of remaining uncovered POIs at the beginning of the t-th round. The greedy algorithm selects the package S_t with the minimum marginal cost-effectiveness, so we must have:

$$\frac{b_{S_t}}{m_t} \le \frac{b_S}{|S \cap R_t|} \le \frac{C_{opt}}{n_t} \tag{5}$$

i.e., $b_{S_t} \le \frac{m_t}{n_t} C_{opt}$. Accumulating over all rounds:

$$C_{greedy} = \sum_t b_{S_t} \le C_{opt} \cdot \sum_t \frac{m_t}{n_t} \tag{6}$$

Number the POIs covered in each round in the order they are covered as $1, 2, \ldots, N$. Let the number of remaining uncovered POIs when the q-th covered POI is covered be n_q. Since at least one POI is covered per round and POIs are removed one by one, $n_q \ge N - q + 1$. Therefore:

$$C_{greedy} \le C_{opt} \cdot \sum_{q=1}^{N} \frac{1}{n_q} \le C_{opt} \cdot \sum_{q=1}^{N} \frac{1}{N - q + 1} = H_N \cdot C_{opt} \tag{7}$$

3 Additional Experimental Results

This section reports the additional experimental results.

3.1 False Negative Rate Based on MIP

To further analyze the gap between different algorithms and the MIP exact solver's determination results, this paper takes the MIP exact solver's determination results as the baseline and counts the false negative rates of the remaining algorithms on instances where "MIP is feasible and not timed out", with specific effects shown in Table S1.

From the above table, it can be seen that GA-LNS has a core advantage in feasibility determination. When $N \le 9$, the false negative rate of GA-LNS always remains 0%, completely consistent with MIP's determination. As the task package scale increases, the false negative rate calculated by the GA-LNS algorithm begins to increase, being 2.54% at $N = 10$, 11.69% at $N = 15$, and 15.79% at $N = 20$. In contrast, the value calculated by Greedy is slightly higher than GA-LNS; Random's false negative rate rises sharply with N, being 1.73% at $N = 5$, already 69.57% at $N = 10$, as high as 99.35% at $N = 15$, and reaching 100% at $N = 20$.

At the same time, the false negatives of GA-LNS are not caused by algorithm defects, but by failing to hit a feasible permutation under a limited search budget; therefore it belongs to not finding a solution, rather than incorrectly proving infeasibility.

Table S1: False negative rate (%) of different algorithms based on MIP.

N	GA-LNS	Greedy	Random
1	**0.0**	**0.0**	**0.0**
2	**0.0**	**0.0**	**0.0**
3	**0.0**	**0.0**	**0.0**
4	**0.0**	*0.19*	**0.00**
5	**0.0**	1.15	*1.73*
6	**0.0**	**0.0**	*2.33*
7	**0.0**	**0.0**	*2.88*
8	**0.0**	0.36	*20.43*
9	**0.0**	1.47	*46.52*
10	**2.54**	4.71	*69.57*
11	**1.22**	**1.22**	*79.88*
12	**4.07**	**4.07**	*95.12*
13	**5.48**	**5.48**	*97.26*
14	**6.99**	**6.99**	*97.9*
15	**11.69**	**11.69**	*99.35*
16	**3.39**	**3.39**	*100.0*
17	**0.0**	**0.0**	*100.0*
18	**2.7**	**2.7**	*100.0*
19	**12.7**	**12.7**	*100.0*
20	**15.79**	**15.79**	*100.0*

3.2 Optimality Gap on Representative Instances

To further analyze the optimality gap of different algorithms for specific instances, this paper selects instances from all 5000 instances based on the number of POI points N in the instance. For example, select one for $N = 1$, one for $N = 2$, and so on, selecting 20 representative instances in total, and processes them with four algorithms respectively, with specific effects shown in Table S2.

Table S2: Latest return time (s) of four algorithms for specific instances

N	Instance	Drone	MIP	GA-LNS	Greedy	Random
1	0	1	**271**	**271**	**271**	**271**
2	1870	2	**2330**	**2330**	**2330**	**2330**
3	3592	3	**2194**	**2194**	*2195*	**2194**
4	3953	4	**2160**	**2160**	*2200*	*2200*
5	1145	1	**3216**	**3216**	3296	*3873*
6	4504	4	**2881**	**2881**	3001	*3723*
7	4765	4	**2769**	**2769**	2930	*3378*
8	4947	4	**3423**	**3423**	3619	*3719*
9	2156	2	**4314**	*4553*	*4553*	-
10	4899	4	**3063**	**3063**	3337	*4594*
11	2754	3	**3973**	**3973**	*4294*	-
12	4693	4	**3550**	**3550**	3910	*5616*
13	4032	4	**3891**	**3891**	*4291*	-
14	3704	3	**5267**	**5267**	*5703*	-
15	3070	3	**5047**	**5047**	*5528*	-
16	4517	4	**5188**	5317	*5688*	-
17	4439	4	**4636**	**4636**	*5198*	-
18	3758	4	**4734**	**4734**	*5309*	-
19	4539	4	**5118**	**5118**	*5758*	-
20	3973	4	**4868**	**4868**	*5549*	-

As shown in the above table, "-" indicates infeasible. Among the 20 representative instances, the solution results of GA-LNS are completely consistent with MIP in 18 instances, with differences only appearing at $N = 9$ with a gap of 239s, and at $N = 16$ with

a gap of 129s. The average Gap of GA-LNS is only 18.4s, with a relative error of about 0.4%. In contrast, the Gap of Greedy generally shows an expanding trend after $N \geq 4$, reaching 681s at $N = 20$, with a relative error of about 14%. This fully verifies the ability of GA-LNS to approach the theoretical optimum.

3.3 Comparison of Solving Time

As the problem scale N continues to increase, the solving time of MIP will gradually increase, causing the problem to be unable to be solved quickly. To reasonably analyze this problem, under the global deadline T_{lim} and battery constraint $E_i^{\max}$, the average solving time of four drones under different algorithms is analyzed, with specific effects shown in Figure S1.

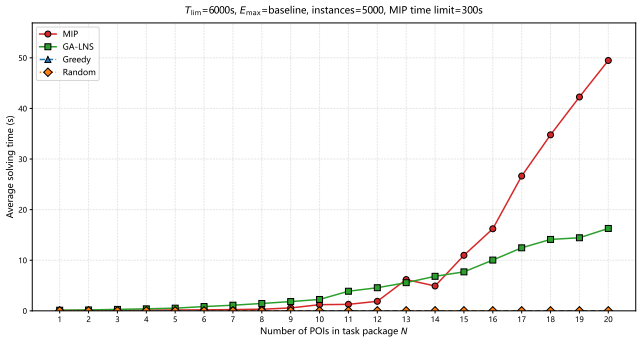


Figure S1: Average solving time of four algorithms under different POI numbers N.

As shown in the figure, the average solving time of the MIP exact solver grows rapidly with the task package scale, approximately showing superlinear growth within the test range: from 0.07s at $N = 1$ to 1.22s at $N = 10$, reaching 10.97s at $N = 15$, and as high as 49.47s at $N = 20$. In contrast, the solving time of GA-LNS shows approximately linear growth: from 0.13s at $N = 1$ to 2.25s at $N = 10$, reaching 7.70s at $N = 15$, and 16.28s at $N = 20$.

The running times of Greedy and Random are both in the millisecond level, but their feasibility ratio or return time $t_{\max}$ deviates significantly, making them unable to undertake strict determination tasks. Combining the two dimensions of time and feasibility, GA-LNS is the only algorithm that possesses both high feasibility and acceptable running time.

3.4 Impact of Battery Capacity Fluctuation on Feasibility Ratio

To investigate the impact of battery capacity constraints on feasibility determination, under the existing deadline constraint, let the battery capacity fluctuate in the range $f = [0.8, 1.2]$ of the original baseline value with a step size of 0.1, and analyze the average feasibility ratio change trend of four types of drones under four algorithms, with specific results shown in Figure S2.

As shown in the figure, when the battery capacity fluctuates in the range $[0.8, 1.2]$, the sensitivity of different grades of drones shows significant differences.

The feasibility ratio of L1 drones is always higher than 90%, and after $f \geq 0.9$, MIP, GA-LNS, and Greedy reach or approach

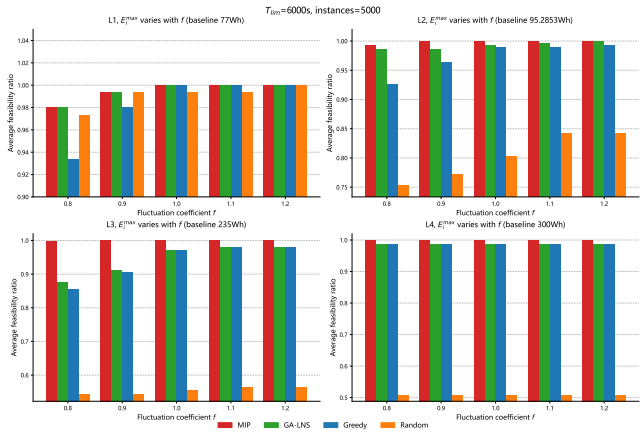


Figure S2: Impact of battery capacity fluctuation on feasibility ratio.

100%, and Random also improves significantly and approaches 100%. This grade has a small task package scale and sufficient battery redundancy. The battery capacity under the baseline configuration is enough to support all task packages, so parameter fluctuations have limited impact on feasibility.

L2 drones are most sensitive to battery capacity. The feasibility ratio of GA-LNS increases from 98.67% at $f = 0.8$ to 100% at $f = 1.2$; Greedy increases from 92.67% to 99.33%; Random increases from 75.33% to 84.33%. The four algorithms form a clear hierarchy in this grade: overall showing a hierarchical trend of MIP not inferior to GA-LNS, GA-LNS better than Greedy, and Greedy better than Random; at the high battery end, some algorithms converge to 100%, indicating that the battery constraint of this grade of drones is in a tight balance state, and its feasibility depends on both algorithm strategy and physical constraints.

The algorithm differences of L3 drones are more obvious in the low battery range, but converge in the high battery range. The feasibility ratio of GA-LNS increases from 87.56% at $f = 0.8$ to 96.89% at $f = 1.0$; Greedy increases from 85.56% to 96.89%, and the two tend to be consistent after $f \geq 1.0$. However, the feasibility ratio of Random remains in the 54% $\sim$ 56% range throughout the fluctuation interval, and hardly improves with the increase of battery. This shows that when the task package scale reaches the order of $N = 15$, the random strategy can no longer compensate for the lack of path planning capability through battery redundancy.

L4 drones show an obvious plateau effect: GA-LNS and Greedy are stable at 98.5% under all fluctuation coefficients f, MIP is stable at 100%, and Random is stable at 50.83%. This indicates that under the baseline battery capacity configuration, this grade of drones is no longer dominated by energy constraints. Relative to battery capacity, its feasibility is more affected by time scheduling and visiting sequence.

3.5 Impact of Deadline Fluctuation on Feasibility Ratio

To investigate the impact of deadline constraints on feasibility determination, under the existing battery constraint, let the deadline

fluctuate in the range $f = [0.8, 1.2]$ of the original baseline value with a step size of 0.1, and analyze the average feasibility ratio change trend of four types of drones under four algorithms, with specific results shown in Figure S3.

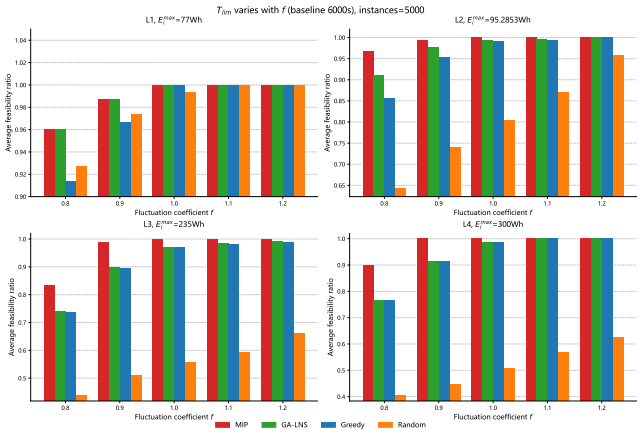


Figure S3: Impact of deadline fluctuation on feasibility ratio.

As shown in the figure, the impact of deadline fluctuation on the four grades of drones presents characteristics completely different from battery capacity fluctuation.

L1 drones are not sensitive to deadlines. The feasibility ratio of GA-LNS at $f = 0.8$ is 96.00%, and reaches 100% after $f \geq 1.0$. Combined with the analysis of Figure S2, it can be seen that the bottleneck of this grade of drones is battery capacity rather than time.

L2 drones are relatively sensitive to deadlines. The feasibility ratio of GA-LNS monotonically increases from 91.00% at $f = 0.8$ to 100% at $f = 1.2$; Greedy increases from 85.67% to 100%; Random increases from 64.33% to 95.67%. The Random curve shows an obvious gradient throughout the fluctuation interval, increasing by more than 30 percentage points from 64.33% to 95.67%. Combined with the phenomenon in Figure S2 that the impact of battery capacity fluctuation is small, while the impact of deadline in Figure S3 is significant, it can be judged that this grade of drones is more sensitive to time scheduling and visiting sequence.

The discrimination of L3 drones is the most significant. The feasibility ratio of GA-LNS increases from 74.00% at $f = 0.8$ to 99.33% at $f = 1.2$, an increase of 25 percentage points; Greedy increases from 73.78% to 98.89%; Random increases from 43.78% to 66.22%, always below 70%. The feasibility ratio of MIP at $f = 0.8$ is 83.33%, with a gap of 9.33 percentage points from GA-LNS's 74.00%, indicating that on the critical task packages of this grade of drones, the search budget of GA-LNS is no longer sufficient to fully approach the theoretical optimum, but it still has significant advantages over Greedy and Random.

L4 drones have the strongest time sensitivity. The feasibility ratio of MIP increases from 90.00% at $f = 0.8$ to 100% at $f = 1.0$; GA-LNS increases from 76.67% to 100% at $f = 1.1$; Greedy increases from 76.67% to 100% at $f = 1.1$; Random increases from 40.33% to 62.33% at $f = 1.2$. Even under the loose time constraint of $f = 1.2$, the feasibility ratio of Random is still less than 65%, indicating that the task package scale of this grade of drones has exceeded the effective working range of the random strategy.

Combining the two sets of sensitivity experiments in Sections above, it can be found that the constraint bottlenecks of the four types of drones have significant differences: The main bottleneck of L1 and L2 drones is battery capacity. These two types of drones have small task package scales, relatively loose time constraints, and battery constraints in a tight balance state. Near the baseline configuration, the main bottleneck of L3 and L4 shifts from battery constraints to deadlines, trip division, and visiting sequence; but this does not mean that battery constraints completely disappear, especially in the combination of low battery/tight time at $f = 0.8$. These two grades are equipped with industrial-grade batteries, with sufficient battery redundancy under the baseline configuration, and their feasibility is mainly affected by deadlines and visiting sequence. This result provides a reference basis for differentiated scheduling of heterogeneous drones in post-earthquake emergency scenarios: for low-end consumer-grade models, battery redundancy should be prioritized; for high-end industrial-grade models, trip division and visiting sequence should be prioritized.

In addition, from the algorithm dimension, GA-LNS always maintains a feasibility determination capability highly consistent with MIP in the two sets of sensitivity experiments, and is significantly better than Greedy and Random. Taking the L3 grade drone as an example, when T_{lim} is compressed to 80% of the baseline value, GA-LNS still maintains a feasibility ratio of 74.00%, while Random has dropped to 40.33%. This shows that GA-LNS has obvious advantages over random/greedy strategies in time-pressured scenarios.

3.6 Verification of Individual Rationality and Incentive Compatibility

In the $M = 100$ grade, randomly select a representative winner and a representative loser, and draw their bid-utility curves. The specific effects are shown in Figure S4.

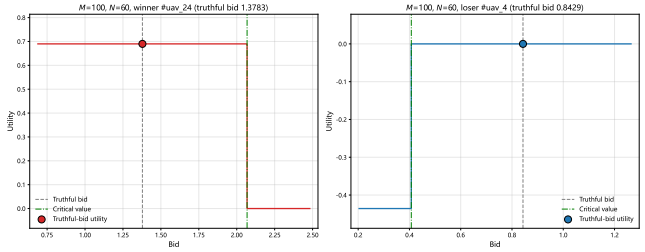


Figure S4: Bid-utility variation diagram of winner and loser.

As shown in the figure, for the winner device (left figure of Figure S4), when the bid is lower than the critical price 2.0682, the device wins and the utility is constant at 0.6899; once the bid exceeds the critical price, the device immediately loses and the utility drops to 0. The truthful bid 1.3783 is on the left side of the critical price, with positive utility, which is a weakly dominant strategy. For the loser device (right figure of Figure S4), the truthful bid 0.8429 has exceeded the critical price 0.4072, the device loses and the utility is 0, indicating that when the bids of others are fixed, the upper limit the market is willing to pay for this device is only 0.4072, lower

than its true cost 0.8429; therefore, truthful bidding and losing is rational, not a mechanism misjudgment; if misreporting a low bid below 0.4072, although it can win, the GCPM payment equals the critical price 0.4072, lower than the true cost 0.8429, and the utility drops to $0.4072 - 0.8429 = -0.4357$, resulting in a loss instead.

Among the 10 sampled auctions, totaling 1000 devices, no IR violations were observed; when performing a one-dimensional deviation scan on individual device bids, no deviation strictly better than truthful bidding was found. This result is consistent with the theoretical IR/IC properties of GCPM.

3.7 Verification of GCPM Approximation Ratio

To verify the tightness of the approximation ratio bound given in Theorem S5, this paper randomly samples 100 independent auction instances in each of the 5 candidate package scale grades, and counts the ratio of the social cost of GCPM to the exact optimal of VCG. The results are shown in Figure S5.

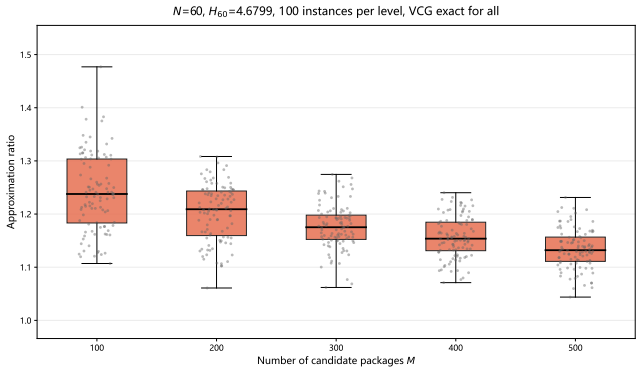


Figure S5: Comparison of GCPM approximation ratio distribution and theoretical bound H_N.

Three points can be seen from the figure. First, the average approximation ratio of GCPM in all grades is significantly lower than the theoretical bound, and the average approximation ratio of GCPM is about 24% $\sim$ 27% of the theoretical bound H_{60}. This shows that although the H_N bound is theoretically loose ($H_{60} = \sum_{k=1}^{60} \frac{1}{k} \approx 4.6799$), the actual performance of GCPM in the experimental scenario of this paper is close to optimal. Second, as the number of candidate packages M increases, the average approximation ratio of GCPM monotonically decreases, and the distribution variance also gradually narrows. The mean decreases from 1.2404 at $M = 100$ to 1.1344 at $M = 500$, a decrease of 8.5%; the standard deviation decreases from 0.0729 to 0.0374, a decrease of 49%. This indicates that the richer the candidate pool, the easier it is for GCPM to find a coverage plan close to optimal. Third, within 500 sample instances, the maximum observed value of the GCPM approximation ratio does not exceed 1.50.

In summary, GCPM achieves a social cost close to the VCG optimal solution in polynomial time while ensuring incentive compatibility and individual rationality. This result verifies the effectiveness of GCPM as a deployable mechanism, and also shows that although the H_N bound given in Theorem S5 is theoretically loose, it does not become a performance bottleneck in actual emergency auction scenarios.